Apache Airflow

CHEAT SHEET · Quick Reference Guide

WHAT IS AIRFLOW?

Apache Airflow is an open-source platform to programmatically **author, schedule and monitor workflows** (DAGs).

Python-based	Define workflows as code using Python.
Extensible	Hundreds of operators and hooks for any tool.
Rich UI	Built-in web interface for monitoring and management.
Strong community	Large open-source ecosystem and Apache project.

CORE CONCEPTS

DAG	Directed Acyclic Graph of tasks. Represents the entire workflow.
Task	A single unit of work. Implemented using Operators.
Operator	Defines what action a task performs (e.g. PythonOperator, BashOperator).
DAG Run	A specific execution instance of a DAG for a particular execution_date.
Schedule	Defines when the DAG should run — cron expression or preset like @daily.
Worker	The process that actually executes the task code.
Connection	Stores external system credentials (DB, API, Cloud services, etc.).
Variable	Key-value store for dynamic configuration shared across DAGs.

AIRFLOW ARCHITECTURE

Components and their roles:

Web Server	Flask-based UI. Manages and monitors DAGs, task status, logs, connections and variables.
Scheduler	Continuously parses DAG files, determines which tasks are ready, and submits them to the executor.
Executor	Decides HOW tasks run: LocalExecutor (same machine), CeleryExecutor (distributed workers), KubernetesExecutor (one pod per task).
Workers	Processes that execute your task code. In Celery mode, these are separate machines pulling from a queue.
Metadata DB	PostgreSQL or MySQL database storing all state: DAG runs, task instances, logs, connections, variables.
Triggerer	Lightweight process that runs deferrable operators asynchronously without blocking a worker slot.

COMMON OPERATORS

`PythonOperator`	Executes a Python callable. Pass python_callable= and op_kwargs= for arguments.
`BashOperator`	Runs a shell/bash command. Use bash_command= parameter.
`EmailOperator`	Sends an email notification via configured SMTP connection.
`BranchPythonOperator`	Returns a task_id to branch to; all other branches are skipped automatically.
`DummyOperator`	No-operation placeholder. Useful for grouping or visual structure in DAGs.
`ShortCircuitOperator`	Skips all downstream tasks if the callable returns False.
`Sensor (BaseSensor)`	Blocks pipeline execution until a condition is True. Configurable poke_interval and timeout.
`KubernetesPodOperator`	Launches a Kubernetes pod to run the task. Ideal for containerised workloads.
`HttpOperator`	Makes HTTP requests to an external API endpoint.

SCHEDULE INTERVALS

`@once`	Run once and never again.
`@hourly`	Run at the start of every hour.
`@daily`	Run once a day at midnight (00:00).
`@weekly`	Run once a week at midnight on Sunday.
`@monthly`	Run on the first day of every month.
`@yearly`	Run on January 1st every year.
`0 0 * * *`	Cron: every day at midnight.
`30 2 * * 1-5`	Cron: every weekday at 02:30 AM.
`*/15 * * * *`	Cron: every 15 minutes.

Cron format: minute hour day-of-month month day-of-week (0–59) (0–23) (1–31) (1–12) (0–6)

DAG FILE EXAMPLE

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime, timedelta

def print_hello():
    print("Hello from Airflow!")

with DAG(
    dag_id="example_dag",
    start_date=datetime(2024, 1, 1),
    schedule_interval="@daily",
    catchup=False,
    default_args={"owner": "data-eng", "retries": 1,
                  "retry_delay": timedelta(minutes=5)},
    tags=["example"]
) as dag:

    hello_task = PythonOperator(
        task_id="hello_task",
        python_callable=print_hello
    )
```

DAG DEPENDENCIES

Set task dependencies using the bitshift operator or set_downstream():

```
# Bitshift operator (recommended)
task1 >> task2 >> task3

# List — multiple downstream tasks
task1 >> [task2, task3]

# Using set_downstream()
task1.set_downstream(task2)
task2.set_downstream(task3)
```

USEFUL CLI COMMANDS

`airflow db init`	Initialise the metadata database (run once on install).
`airflow webserver`	Start the web server (default port 8080).
`airflow scheduler`	Start the scheduler process.
`airflow dags list`	List all available DAGs.
`airflow dags trigger`	Trigger a DAG run manually.
`airflow tasks list`	List all tasks in a DAG.
`airflow tasks test`	Test a single task without recording to DB.
`airflow users create`	Create a new Airflow user account.

COMMON AIRFLOW COMPONENTS

DAGs Folder	Directory where DAG Python files are stored. Scheduler scans this folder continuously.
Logs	Task instance logs stored in configured location (local filesystem, S3, GCS, etc.).
XCom	Cross-communication: exchange small data values between tasks using push/pull API.
Plugins	Extend Airflow with custom operators, hooks, sensors and UI views.
Pools	Limit the number of tasks running concurrently on shared resources.
Variables	Store global config values as key-value pairs accessible from any DAG.
Connections	Named credentials for external systems: databases, APIs, cloud providers.

BEST PRACTICES

- ✓ **Idempotent DAGs** — Running a DAG multiple times with the same inputs must produce the same result. Avoid side effects that accumulate.
- ✓ **Use catchup=False** — Prevents Airflow from backfilling all missed runs on first deploy. Enable only when historical fills are explicitly needed.
- ✓ **No heavy code in DAGs** — Avoid DB calls or slow imports at DAG file level. The scheduler parses all DAG files frequently — slow parsing degrades performance.
- ✓ **Variables & Connections** — Never hardcode credentials or env-specific values. Use Airflow Variables for config and Connections for service auth.
- ✓ **Set retries** — Set retries=2 and retry_delay=timedelta(minutes=5) as a baseline to handle transient failures gracefully.
- ✓ **Meaningful IDs** — Use descriptive task_id and dag_id names. Add tags=["team","env"] to organise DAGs in the UI.
- ✓ **Monitor with alerts** — Configure email or Slack alerts on failure. Use SLAs to catch delayed pipelines early.

Apache Airflow — Detailed Reference

In-depth explanations for every section on Page 1

ARCHITECTURE — DEEP DIVE

Web Server

Built on Flask and Gunicorn. Serves the Airflow UI on port 8080 by default. Lets you browse DAGs, inspect individual task runs, read logs, manage connections/variables, and manually trigger or clear runs. The web server reads directly from the Metadata DB — it does not control task execution.

Scheduler

The central orchestrator. It continuously parses every Python file in the DAGs folder, resolves task dependencies for each DAG Run, and publishes ready tasks to the executor. The scheduler uses the execution_date concept to handle backfills and concurrent runs. In Airflow 2.x, multiple scheduler instances can run in HA (High Availability) mode.

Executor

An abstraction layer that decides how tasks are physically run. • SequentialExecutor — one task at a time, SQLite only, dev/testing only. • LocalExecutor — parallel tasks on one machine using subprocesses. • CeleryExecutor — distributes tasks across many worker machines via Redis or RabbitMQ. • KubernetesExecutor — launches a fresh Kubernetes pod per task; no persistent workers needed.

Workers

The processes that import and execute your actual task code. In CeleryExecutor mode, workers are standalone processes (often on separate VMs) that subscribe to a task queue. Each worker picks up tasks, executes them, reports the result, and updates the Metadata DB via the scheduler.

Metadata Database

The single source of truth for all Airflow state. Stores: DAG definitions and parsing results, DAG Run history, Task Instance states (queued, running, success, failed…), XCom values, Connections, Variables, User accounts. Use PostgreSQL or MySQL in production. SQLite is only suitable for local development with SequentialExecutor.

Triggerer

Introduced in Airflow 2.2 for deferrable operators. Instead of a worker holding a slot while waiting (e.g. polling S3 every 30s), the task defers itself to the Triggerer. The Triggerer runs many async checks in one thread, freeing worker slots for real computation. This dramatically reduces resource usage for sensor-heavy pipelines.

TASK STATES — LIFECYCLE

none	No task instance recorded yet. Initial state before any scheduling.
scheduled	Scheduler has identified this task as due to run based on its dependencies and schedule.
queued	Task has been submitted to the executor and is waiting for a free worker slot.
running	A worker is actively executing the task code right now.
success	Task completed without raising an exception. All downstream tasks can now proceed.
failed	Task raised an exception. If retries remain, transitions to "up_for_retry" then "retry".
retry	Task failed but has retries remaining. Will be re-queued after retry_delay has passed.
skipped	Downstream task skipped by BranchPythonOperator or ShortCircuitOperator returning False.
upstream_failed	A task this task depends on has failed, so this task cannot run.
up_for_reschedule	Used by Sensors in reschedule mode: task is paused between poke attempts to free the worker.

OPERATORS — WHEN & HOW TO USE

PythonOperator

Most flexible operator. Use python_callable= to point to any Python function. Pass arguments via op_kwargs={"key": "value"}. The function runs inside the worker process, so it has access to all installed packages. Return value is automatically pushed as an XCom.

BashOperator

Runs shell commands or scripts. Use bash_command="your_script.sh". Environment variables can be injected via env={"KEY": "value"}. Exits non-zero means task failure. Combine with Jinja templating: bash_command="echo {{ ds }}" to inject the execution date.

BranchPythonOperator

Must return a task_id string (or list of task_ids) to follow. All other branches are automatically marked as skipped. Downstream joins must use trigger_rule="none_failed_min_one_success" to handle skipped branches correctly.

Sensors (BaseSensor)

Two modes: poke (default) — worker slot held the entire time, checks every poke_interval seconds; reschedule — worker slot released between checks, far more efficient for long waits. Always set a timeout to prevent infinite blocking. Common sensors: FileSensor, S3KeySensor, HttpSensor, SqlSensor.

KubernetesPodOperator

Launches a Docker container in Kubernetes per task. Ideal for: tasks with conflicting Python dependencies, tasks needing GPU/special hardware, or total environment isolation. Specify image=, cmds=, arguments=, env_vars= and resources=. Logs are streamed back to Airflow.

ShortCircuitOperator

If the callable returns False, the operator marks itself and all downstream tasks as skipped, effectively stopping that branch. If True, execution continues normally. Useful for conditional ETL (e.g. only run on Mondays, only if new data exists).

XCOM, POOLS, VARIABLES & CONNECTIONS

XCom (Cross-Communication)

Allows tasks to pass small values to each other. A task pushes a value with xcom_push(key, value) or simply by returning a value from the callable. A downstream task pulls it with xcom_pull(task_ids="upstream_task", key="return_value"). XCom is stored in the Metadata DB — keep values small (< 48 KB). For large datasets, write to S3/GCS and pass the path via XCom instead.

Pools

Named slots that limit how many tasks can run concurrently on a shared resource. Example: if your database handles max 5 connections, create a pool "db_pool" with 5 slots and assign relevant tasks pool="db_pool". Tasks beyond capacity queue automatically. Manage pools in the Airflow UI under Admin > Pools.

Variables

Global key-value store in the Metadata DB. Access in DAGs with Variable.get("my_key", default_var="fallback"). Supports JSON: Variable.get("config", deserialize_json=True). Mark sensitive variables as secret so they are masked in the UI. Prefer environment variables or secrets backends for credentials at scale.

Connections

Named credential bundles for external systems. Each Connection has a conn_id, conn_type, host, port, login, password and extras (JSON). Hook classes (PostgresHook, S3Hook, HttpHook) look up connections by conn_id automatically. Store connections in: Airflow UI, environment variables (AIRFLOW_CONN_=uri), or secrets backends like AWS Secrets Manager, HashiCorp Vault, or GCP Secret Manager.

BEST PRACTICES — EXPLAINED

Idempotency

An idempotent DAG produces the same outcome regardless of how many times it runs for a given execution_date. Achieve this by using INSERT ... ON CONFLICT, MERGE/UPSERT, or writing to date-partitioned paths (s3://bucket/date={{ ds }}/). Never use APPEND-only writes unless explicitly managing deduplication.

catchup=False

When you first deploy a DAG with a past start_date, Airflow will schedule and run every missed interval unless catchup=False. For production pipelines that should only run going forward, always set catchup=False. Use airflow dags backfill for intentional historical runs.

Avoid top-level code

The Scheduler imports every DAG file every few seconds (default 30s). Any code at module level runs repeatedly. This means: no database queries, no HTTP calls, no heavy computations at import time. Move all logic inside callables. Use lazy evaluation and caching for any setup that must happen at module level.

retries and retry_delay

Production pipelines encounter transient failures: network timeouts, rate limits, brief database unavailability. Setting retries=3, retry_delay=timedelta(minutes=5) gives your tasks three additional attempts before marking as failed. Use exponential backoff with retry_exponential_backoff=True for APIs that rate-limit.

Monitoring and Alerting

Configure email_on_failure=True or a custom on_failure_callback to send Slack/PagerDuty alerts. Set sla=timedelta(hours=2) on critical tasks — Airflow will alert if the task misses its SLA window. Use the built-in DAG-level SLA miss callback for pipeline-wide monitoring. Integrate with Datadog, Prometheus, or Grafana via StatsD metrics.